Class and Objects: A *class* is a user-defined blueprint (e.g., `class Student { String name; int age; }`), and an *object* is an instance created from it (e.g., `Student s1 = new Student(); s1.name = "John";`).

Four Pillars of OOP:

- **Encapsulation:** Wrapping data and methods in a single unit (class) using private variables with public getters/setters (e.g., `private int balance; public getBalance()`).
- **Inheritance:** Creating a new class from an existing one using `extends` (e.g., `class Dog extends Animal { }` reuses Animal's properties).
- **Polymorphism:** Same method behaving differently, e.g., method overloading (`add(2,3)` and `add(2,3,4)`) or overriding (`sound()` makes "bark" for Dog, "meow" for Cat).
- **Abstraction:** Hiding implementation details using `abstract` classes or interfaces (e.g., `abstract class Vehicle { abstract void start(); }` - each vehicle starts differently).

Constructor and types: A constructor initializes objects automatically when created (e.g., `new Car()`). **Types:** *Default* (no arguments: `Car() { }`), *Parameterized* (accepts values: `Car(String color) { this.color = color; }`), *Copy* (copies another object: `Car(Car c) { this.color = c.color; }`).

Method Overloading: Multiple methods with same name but different parameters. **Ways in Java:** 1) Different *number* of parameters: `void show(int a)` vs `void show(int a, int b)`. 2) Different *data types*: `void show(int a)` vs `void show(double a)`. 3) Different *order* of parameters: `void show(int a, String b)` vs `void show(String a, int b)`.
Example: `System.out.println()` works for int, String, double, etc.